Accelerating Super-Resolution CNN Inference via GPU Spatial Reduction: A Cross-Platform Study on Unified and Discrete Memory Architectures

Anonymous Author(s)
Affiliation withheld for double-blind review

***Abstract*—FSRCNN's final deconvolution layer is both its computational bottleneck and a correctness hazard: naive multi-threaded CPU accumulation races and corrupts output non-reproducibly. We offload it to isolated per-channel CUDA kernels followed by a parallel reduction, eliminating races by construction, and evaluate on two architecturally distinct platforms: an NVIDIA GB10 Grace Blackwell Superchip with unified memory, and an Intel i9-14900K desktop with an RTX 4090 over PCIe. On both, CPU and GPU spatial-reduction output is byte-identical to a verified deterministic reference at every configuration tested, and the GPU output is bit-identical across the two platforms despite differing host instruction set architecture and GPU microarchitecture. The warp-aligned 32-by-8 grid is fastest on both, and per-kernel profiling localizes the entire gain to the deconvolution kernel, as the memory-coalescing explanation predicts. Speedups over each platform's best correctness-verified CPU configuration reach 1.46 times on the GB10, at 31.5 frames per second, and 1.63 times on the RTX 4090, at 27.6 frames per second. Profiling further identifies an architectural boundary for the cost of the pattern: determinism costs 4.6 percent of GPU layer-8 time on the GB10 but 0.6 percent on the RTX 4090, depending on whether the private buffer fits in last-level cache, while device-to-host transfer runs the opposite way and is 4.74 times faster on unified memory.**

***Index Terms*—Super-resolution, FSRCNN, GPU offloading, CUDA, spatial reduction, Grace Blackwell, asymmetric multicore, bit-exactness.**

I. Introduction

Edge and workstation-class AI hardware increasingly adopt SoCs in which multicore CPUs and GPUs co-exist on high-speed interconnects [20], [21]. Super-resolution CNNs have moved in step. Early formulations [9] gave way to FSRCNN [2], which defers feature extraction to low-resolution space and upscales only at the final deconvolution layer.

That final stage (Layer 8) carries a disproportionate cost: for $2\times$ upscaling it applies a 9×9 kernel across 56 channels and accumulates them into one output plane. Parallelizing it on a multicore CPU poses a dilemma. Naive shared accumulation races and corrupts output non-reproducibly [1], while mutexes or atomics bring prohibitive contention. Privatize-then-reduce avoids both by writing per-channel private buffers and reducing them in a separate deterministic pass, at the price of 43.3 MiB of extra traffic.

We offload that deconvolution and reduction to CUDA while parallelizing Layers 1–7 across CPU cores. Our contributions are four. *(i) A GPU cost-profile shift:* privatize-then-reduce is a classical pattern, not a new algorithm, but moving it to the GPU changes its cost qualitatively. The private-buffer traffic costing 8–18% of CPU wall time falls to 4.6% (GB10) and 0.6% (RTX 4090) of GPU Layer 8 time while output stays bit-exact. We frame this as bandwidth and parallelism rather than capacity, since our primary platform shares one pool between CPU and GPU, so the buffer never moves; only the computation over it does. *(ii) Warp-aligned grid configuration:* a 32×8 block maximizes coalescing for the deconvolution's row-major access, cutting kernel time 23.4% and 17.4%, with per-kernel profiling localizing the entire gain to the deconvolution as the explanation predicts. *(iii) Cross-platform determinism:* GPU output is byte-identical to the CPU baseline at every configuration tested on both platforms and bit-identical *across* them, spanning two host ISAs and two GPU generations, at $1.46\times$ and $1.63\times$ over each platform's best deterministic CPU configuration. *(iv) An architectural boundary for the pattern's cost:* the reduction is bandwidth-bound, and its cost depends on whether the private buffer fits last-level cache. It is near-free on the RTX 4090, whose L2 holds it, and $10\times$ costlier on the GB10, which streams it at 95% of memory peak. Device-to-host transfer runs the opposite way, $4.74\times$ faster on unified memory.

II. Background and Baseline Architecture

A. Related Work

CNN super-resolution progressed from early formulations [9] to FSRCNN [2], increasingly deployed on heterogeneous CPU+GPU SoCs [20], [21]. Closest to ours, Annisa et al. [1] characterize the same Layer 8 race on an Orange Pi 5 with the same Suzie sequence, finding corruption worse under LITTLE-core affinity; their aim is characterizing that corruption rather than remedying it, and we do not compare speedups given the difference in scale. Privatize-then-reduce is classical; our contribution is adopting it as a race-free CPU baseline and moving it to the GPU, which changes its cost profile while preserving bit-exactness. CNN inference scales well on ARM big.LITTLE under proper scheduling [3], so our GB10 baseline is non-trivial. Libraries such as cuDNN [12] target convolution broadly but not this race-free layout. Discrete-GPU PCIe overhead is well documented [7], motivating the unified-versus-discrete comparison, and recent work characterizes unified physical memory directly [21]. Our RTX 4090 scheduling collapse relates to work on asymmetric-

CPU scheduling [4], [5]; there we contribute an observation, not a mechanism.

B. FSRCNN Architecture

FSRCNN consists of eight layers: Feature Extraction (Layer 1), Shrinking (Layer 2), four Mapping layers (Layers 3–6), Expanding (Layer 7), and Deconvolution (Layer 8). For an input sequence at 176×144 scaled by $2\times$, Layers 1–7 all operate in low-resolution space and produce 56 feature maps. Layer 8 then does the upscaling. It takes those 56 channels, applies a 9×9 transposed convolution [15] with stride 2, and reduces them into a single 352×288 YUV luminance frame.

C. Baseline Variants

We compare three variants. **V0 (naive OpenMP CPU)** has Layer 8 worker threads accumulate deconvolution outputs directly into one shared buffer `img_fltr_8`; with more than one thread, overlapping kernel footprints cause unsynchronized read-modify-write races and output corruption, the class of bug that dynamic detectors such as ThreadSanitizer [13] exist to catch. **V1 (spatial reduction, CPU)** eliminates the race with a private buffer all_tmp$[56 \times 352 \times 288]$ of doubles (43.3 MiB), each channel writing its own isolated offset, followed by a second pass summing the 56 channels per pixel. Both passes are parallel but along different axes: deconvolution over channels, reduction over output pixels. The reduction needs no synchronization because each output pixel is written by exactly one thread, and stays deterministic because that thread accumulates its 56 summands in fixed channel order regardless of scheduling. **V2 (GPU spatial reduction, proposed)** keeps Layers 1–7 on OpenMP CPU threads and offloads the Layer 8 deconvolution and reduction to CUDA.

III. GPU Spatial Reduction Implementation

A. Hardware Targets: GB10 and RTX 4090

Our primary platform is the ASUS Ascent GX10 [17] with the NVIDIA GB10 Grace Blackwell Superchip [16], whose CPU cluster uses Arm's DynamIQ big.LITTLE technology [18]. Section VI cross-validates on an architecturally distinct second platform: an Intel Core i9-14900K desktop with an RTX 4090 (Ada Lovelace [19]), in which CPU and GPU memory are physically separate and PCIe-connected rather than sharing the GB10's unified NVLink-C2C pool. Table I compares them.

TABLE I
HARDWARE SPECIFICATIONS: GB10 (GX10) VS. RTX 4090 DESKTOP

Component	GB10 (ASUS GX10)	RTX 4090 Desktop
CPU	20-core ARM v9.2-A DynamIQ 10× X925 P + 10× A725 E @ 3.90/2.81 GHz, no SMT	Intel Core i9-14900K 8 P-core (w/HT) + 16 E-core 24C/32T, up to 6.0 GHz
GPU	NVIDIA Blackwell, `sm_121`	NVIDIA Ada Lovelace, `sm_89`
GPU Cores	6,144 CUDA, 48 SMs, Gen5 Tensor	16,384 CUDA, 128 SMs [19]
Memory	128 GB LPDDR5x, *unified*	64 GB DDR (63 GB OS-visible) + 24 GB GDDR6X, *discrete*
Interconnect	NVLink-C2C (5× PCIe 5.0)	PCIe (CPU ↔ GPU)
CUDA	13.0, Driver 580.159.03, `V13.0.88`	13.0, Driver 580.65.06, `V13.0.48`
GCC	13.3.0 (Ubuntu 24.04)	13.3.0 (Ubuntu 24.04)
OS	Ubuntu 24.04 LTS (aarch64)	Ubuntu 24.04 LTS (x86_64)

B. CUDA Kernel Design and Warp Alignment

We implement Layer 8 as two hand-written CUDA kernels rather than using a general-purpose library such as cuDNN [12]. We do not claim this beats a cuDNN-based deconvolution. That comparison is untested, and a well-tuned library kernel could plausibly win. Our claim is narrower: writing the pair by hand gives direct control over the private-buffer-plus-reduction layout central to our correctness argument (Section VII).

Deconvolution kernel. Each thread block processes a 2D tile of the input feature map. For each channel $c \in [0, 55]$, threads compute the 9×9 transposed convolution and write into an isolated channel plane of `d_all_tmp`, indexed $c \times (H_{out}W_{out}) + yW_{out} + x$.

Spatial reduction kernel. One thread per output pixel (y, x) iterates all 56 planes, sums them, and adds the Layer 8 bias:

$$\text{output}[y, x] = \left(\sum_{c=0}^{55} \text{d\_all\_tmp}[c, y, x]\right) + \text{bias}_8 \qquad (1)$$

Channel writes are isolated during deconvolution and read-only during reduction, so execution is race-free and deterministic by construction.

NVIDIA GPUs execute threads in SIMD groups of 32 called *warps* [6]. A standard 16×16 block places only 16 threads along the row-major X dimension, so consecutive warps straddle non-contiguous rows. Since warp and block sizing has a first-order effect on SIMT throughput [14], we evaluate a 32×8 block (256 threads): setting $B_x = 32$ guarantees all 32 threads of a warp access contiguous doubles along one image row, maximizing coalescing.

IV. Experimental Evaluation

A. Evaluation Methodology

The workload is 150 frames of the Suzie sequence ($176\times144 \rightarrow 352\times288$, YUV420p). Every configuration ran 7 times: run 1 discarded as warmup, runs 2–7 recorded as mean ± SD. CPU binaries use `gcc -fopenmp -O3` (no `-march=native` or `-ffast-math`) with plain `#pragma omp parallel for`, so placement follows `libgomp`'s static default; the GPU binary uses `nvcc -O3 -arch=native`. Both platforms run GCC 13.3.0 and CUDA 13.0. All arithmetic is double precision, matching the CPU baseline so the bit-exact comparison is as strict as possible; FP32 would admit rounding divergence even for a logically equivalent computation (throughput consequence in Section VI).

Reference file: we run V1 single-threaded once to generate a deterministic reference. Since all three variants implement the identical algorithm and differ only in execution strategy, this is an internal determinism check; Section IV-C scores reconstruction separately. Correctness is assessed by byte comparison (`cmp -l`, diff_bytes $= 0$ for V1 against V2 at every grid configuration) and by PSNR/SSIM [8] computed with `ffmpeg`; V1/V2's PSNR$= \infty$ is logically equivalent to the byte comparison, so we report it as *bit-exact*. PSNR/SSIM in

Table II are all-component averages, so the untouched chroma pulls them upward: the RTX 4090's worst V0 case reads 23.0 dB/0.845 all-component but 21.5 dB/0.767 on luminance alone. Those figures *understate* the corruption, and we give luminance-only numbers wherever its magnitude matters.

V0 degrades non-monotonically from its data race: 13.8 dB on the GB10 (4 threads), and 68.6 dB/0.99996 on the RTX 4090 (4 threads) before collapsing to 23.0 dB at 32 threads. The RTX case is the instructive one. A race costing only 68.6 dB is invisible to casual inspection, yet it is just as nondeterministic as the 13.8 dB case, arguing for determinism by construction rather than trusting that runs "look fine." Three consecutive V0 runs at 32 threads corrupted 4.75, 5.01 and 5.04 million of 22.8 million bytes (21.34, 21.46, 21.63 dB PSNR-Y): never the same output twice. Fig. 1 shows one frame.

Fig. 1. Frame 26. Panel (b) is one representative V0 run of three – V0 is the CPU-only naive baseline; no GPU computation is involved. It uses 32 OpenMP threads on the Intel i9-14900K host CPU of the second (discrete-GPU) desktop platform (21.5 dB PSNR-Y / 0.767 SSIM against (a)); the scanline artifacts come from unsynchronized writes to `img_fltr_8`. Panels (c) and (d) are pixel-identical to (a) at every configuration tested.

B. Performance Scaling & Accuracy Benchmark

Table II presents results for both platforms across thread counts, accuracy, and GPU grid configurations.

C. Reconstruction Quality Against a High-Resolution Reference

Since no high-resolution ground truth exists in the pipeline, we constructed one: Suzie was bicubic-downsampled to 88×72, reconstructed at scale 2, and scored against the original, with bicubic upscaling of the same input as comparator. We report luminance only, since only the Y plane passes through the network. FSRCNN reaches 37.86 dB PSNR-Y and 0.9665 SSIM-Y against bicubic's 35.69 dB and 0.9532, a gain of 2.17 dB over 150 frames. This is one sequence at one scale factor, a sanity check rather than a claim about FSRCNN generally [2].

D. The Cost of Determinism, Quantified

Table II quantifies the CPU-side cost directly, as the wall-time delta between V0 (racy, no privatization) and V1 (privatized) at matched thread counts: $+18.2\%$, $+17.6\%$ and $+9.1\%$ at 1, 4 and 20 threads on the GB10, and $+9.2\%$ and $+7.8\%$ at 1 and 4 threads on the RTX 4090. This delta isolates privatization rather than a difference in how much work is threaded. V0 and V1 parallelize the same deconvolution, and V1's reduction pass is itself parallel over output pixels (Section II-C). What the delta measures is writing and re-reading 43.3 MiB of private buffers, not serializing a stage. Determinism therefore costs 8–18% of CPU wall time. The GB10 points hint the cost falls once the machine saturates, but 1 and 4 threads are nearly identical (18.2% vs. 17.6%), making this a two-level effect at best rather than a trend.

On the GPU, per-kernel instrumentation (Section IV-E) puts the same cost in comparable units. The reduction pass is the GPU analogue of privatization overhead, and it accounts for 4.6% of the Layer 8 event window on the GB10 and 0.6% on the RTX 4090. Those shares are of Layer 8 time rather than of wall time. Put on the same basis as the CPU's 8–18%, the reduction costs 0.55% and 0.05% of end-to-end wall time, which is where the size of the shift actually shows. Peak device memory stays at 44.3 MiB including allocator padding, under 0.1% of either pool; we call this a bandwidth-and-parallelism result rather than a capacity one because on the GB10 there is no separate VRAM for the buffer to move into. The gap between the two GPUs is architectural, not algorithmic: the reduction is bandwidth-bound and the RTX 4090's L2 holds the entire private buffer. Determinism by construction is thus cheapest when cache capacity exceeds the privatization working set. That gives a design rule for applying the pattern to larger networks: overhead scales with how far the buffer overflows last-level cache, not with channel count.

E. Fine-Grained GPU Timing Breakdown

We instrumented CUDA Event profiling inside the GPU binary and ran the optimal configuration (32×8_256) six times on each platform (Table III).

Why we report no host-to-device figure. The host-side counter `h2d_ms` cannot be read as transfer time, and we exclude it. Those copies are issued *inside* the window `gpu_l8_ms` measures, and a blocking copy cannot begin until the previously launched kernel drains, so the counter charges kernel waiting to transfer. The sweep makes this plain: `h2d_ms` shifts by 167 ms on the GB10 when only the *kernel block shape* changes, though the transferred byte count is identical throughout. A counter that moves with block shape at fixed byte count is not measuring bytes. We report the per-kernel decomposition instead.

Two results stand out, pointing in opposite directions.

The reduction kernel costs 26.32 ms on the GB10 but 2.62 ms on the RTX 4090, a $10.0\times$ gap far larger than any general compute advantage (the deconvolution differs by $1.6\times$). The reduction is purely bandwidth-bound, reading the whole 43.31 MiB private buffer once per frame, 6.81 GB over 150 frames: 259 GB/s on the GB10, about 95% of the 273 GB/s peak of its LPDDR5x pool [17], and 2,597 GB/s on the RTX 4090, over $2.5\times$ that card's 1,008 GB/s of GDDR6X. Correcting for per-launch instrumentation overhead (Table III) moves the GB10 figure to 267 GB/s, still below peak, and pushes the RTX 4090 further above its own. A DRAM-resident working set cannot produce that. The natural explanation is the RTX 4090's 72 MB L2, which the buffer fits inside, so the reduction is served largely from cache while the GB10 streams

TABLE II
KEY CONFIGURATIONS ON BOTH PLATFORMS: PERFORMANCE AND DEVIATION FROM EACH PLATFORM'S VERIFIED DETERMINISTIC REFERENCE. WALL TIMES ARE MEAN ± SD OVER THE 6 MEASURED RUNS OF A 7-RUN REPETITION.

Variant	Configuration / Threads	Wall Time (ms)	Speedup vs. Serial	Speedup vs. Best CPU	Dev. from Ref. (dB)	SSIM	GPU L8 Kernel (ms)
GB10 (ASUS GX10), unified memory; CPU thread ladder 1–20							
CPU V0 (Naive)	1 Thread	$18{,}240.50 \pm 228.82$	1.00×	N/A	bit-exact	bit-exact	N/A
CPU V0 (Naive)	4 Threads (worst)	$7{,}074.33 \pm 90.21$	2.58×	N/A	13.785	0.518246	N/A
CPU V0 (Naive)	20 Threads (Max, fastest)	$6{,}346.33 \pm 32.60$	2.87×	N/A	25.751	0.838930	N/A
CPU V1 (Baseline)	1 Thread (Serial)	$21{,}552.33 \pm 298.07$	1.00×	N/A	bit-exact	bit-exact	N/A
CPU V1	4 Threads (cf. RTX optimum)	$8{,}316.50 \pm 58.06$	2.59×	N/A	bit-exact	bit-exact	N/A
CPU V1 (Max Cores)	20 Threads (fastest verified)	$6{,}921.83 \pm 82.49$	3.11×	N/A	bit-exact	bit-exact	N/A
GPU V2 (Hybrid)	Grid 16×16_256 (baseline)	$4{,}935.67 \pm 36.30$	4.37×	1.40×	bit-exact	bit-exact	696.51 ± 1.78
GPU V2 (Best Grid)	Grid 32×8_256	$4{,}756.33 \pm 30.71$	4.53×	1.46×	bit-exact	bit-exact	533.19 ± 2.58
RTX 4090 desktop, discrete memory; CPU thread ladder 1–32, 4-thread GPU backdrop							
CPU V0 (Naive)	1 Thread	$20{,}973.50 \pm 36.30$	1.00×	N/A	bit-exact	bit-exact	N/A
CPU V0 (Naive)	4 Threads (fastest)	$8{,}207.67 \pm 247.16$	2.56×	N/A	68.559	0.999957	N/A
CPU V0 (Naive)	32 Threads (Max, worst)	$26{,}388.67 \pm 664.00$	0.79×	N/A	23.006	0.845283	N/A
CPU V1 (Baseline)	1 Thread (Serial)	$22{,}904.17 \pm 86.90$	1.00×	N/A	bit-exact	bit-exact	N/A
CPU V1 (Best)	4 Threads (fastest verified)	$8{,}844.17 \pm 132.68$	2.59×	N/A	bit-exact	bit-exact	N/A
GPU V2 (Hybrid)	Grid 16×16_256 (baseline)	$5{,}579.00 \pm 166.61$	4.11×	1.59×	bit-exact	bit-exact	437.06 ± 3.86
GPU V2 (Best Grid)	Grid 32×8_256	$5{,}435.50 \pm 148.01$	4.21×	1.63×	bit-exact	bit-exact	360.92 ± 7.22

Speedup vs. Serial: CPU rows against their own variant's 1-thread time; GPU V2 has none of its own, so its denominator is CPU V1's serial time on that platform (Section V-A also reports it against the faster bit-exact V0 serial time). *Speedup vs. Best CPU*: GPU V2 only, against CPU V1's fastest correctness-verified configuration. *Dev./SSIM* are against each platform's own deterministic reference; "bit-exact" is logically equivalent to `cmp -l`, not an independent quality measurement, and figures are all-component averages (Section IV-A). *GPU L8 Kernel*: mean ± SD over 6 runs of the profiling sweep (Section IV-E); kernel times for the three remaining grid configurations are given in Section VI-C.

TABLE III
LAYER 8 PROFILING BREAKDOWN, 32×8_256, 150 FRAMES (MS, MEAN ± SD OVER 6 RUNS)

Pipeline Stage	GB10 (unified)	RTX 4090 (discrete)
CPU Layers 1–7	$3{,}392.82 \pm 21.62$	$4{,}624.50 \pm 70.96$
GPU Layer 8, event window	533.19 ± 2.58	360.92 ± 7.22
↪ `deconv_kernel` (56×)	424.07 ± 0.76	261.90 ± 3.09
↪ `spatial_reduction` (1×)	26.32 ± 1.02	2.62 ± 0.01
↪ in-window transfer, gaps	127.65 ± 4.10	152.14 ± 0.36
Device-to-Host	3.46 ± 0.06	16.42 ± 0.49
Unaccounted (launch, file I/O)	821.20	469.33
Total Wall Time	$4{,}750.67 \pm 30.40$	$5{,}471.17 \pm 76.17$

Indented rows decompose the event window above them and are not additional wall-time slices. They come from a separately instrumented binary whose event recording inflates that window by 8.4% (GB10) and 15.4% (RTX 4090). That overhead is incurred *per launch*, so it is concentrated almost entirely in the 8,400 `deconv_kernel` launches rather than the 150 reduction launches: dividing the inflation by the launch count gives 5.34 µs (GB10) and 6.64 µs (RTX 4090), two independent platforms agreeing to within an order. The reduction figure is therefore inflated by only about 0.80 ms on the GB10, some 3%, and deflating the whole window uniformly instead would wrongly place the reduction above its platform's DRAM peak. The production `h2d_ms` counter is omitted (see text).

from unified memory near hardware peak. We offer this as the reading most consistent with the measured bandwidths, not a profiled fact, having captured no L2 counters (Section VII).

Device-to-host transfer runs the other way: 3.46 ms against 16.42 ms, a 4.74× advantage for unified memory on identical payloads (35.1 vs. 7.4 GB/s effective). This is the cleanest unified-versus-discrete contrast in our data, and unlike `h2d_ms` it is measured outside the event window, after `cudaEventSynchronize`, with no kernel execution to conflate it with.

Peak device memory stayed bounded at 45,382 KiB (≈ 44.3 MiB) on both platforms across every configuration, about 1 MiB above the 43.3 MiB theoretical minimum (Section II-C).

V. DISCUSSION AND KEY INSIGHTS

A. Speedup vs. Maxed-Out CPU Cores, and Warp Alignment

We report speedup against CPU V1 rather than V0 at multi-thread configurations: V1 is race-free at every thread count, while V0 is only *incidentally* correct at 1 thread and degrades thereafter. V0 is in fact the raw wall-time winner at 20 threads (6,346.33 ms, which would give only 1.33×), but its output there is not verified correct; we state the number rather than omit it. GPU V2 (32×8_256) reaches 4,756.33 ms (31.5 FPS), a 1.46× speedup over CPU V1's 20-thread configuration (6,921.83 ms, the fastest correctness-verified CPU result).

The serial comparison needs more care, since the variants share no serial baseline. At 1 thread V0 is bit-exact and faster than V1 (18,240.50 vs. 21,552.33 ms) because it skips the privatization V1 pays for, which makes it the fastest *bit-exact* serial baseline available: 3.84× (GB10) and 3.86× (RTX 4090). Against V1's serial time the figures rise to 4.53× and 4.21×, inflated by the slower denominator, so we lead with 3.84×. The baseline is not a strawman: CNN inference scales well on ARM big.LITTLE clusters when properly scheduled [3], and the GB10's cores scale smoothly across the full ladder with none of the collapse the RTX 4090 shows past 8 threads.

Shifting from the default 16×16 grid (696.51 ± 1.78 ms) to the warp-aligned 32×8 grid (533.19 ± 2.58 ms) cuts kernel time 23.4%, a gap far outside run-to-run variation (SDs of 1.78 and 2.58 ms). We attribute it to coalescing: a 32-thread-wide block lets every thread in a warp issue into one transaction, whereas a 16-wide block splits across several. The per-kernel breakdown tests this more sharply than wall time can. Block shape affects `deconv_kernel` only, since the reduction is launched one-dimensionally. If coalescing is the mechanism,

the whole gain must therefore land in the deconvolution. It does: `deconv_kernel` drops 24.9% (564.74 to 424.07 ms) while the reduction moves +0.9% (26.08 to 26.32 ms), inside its own spread.

B. Transfer Cost and Amdahl's Law

Bounded from above, transfer is not a bottleneck on either platform. On the GB10 everything inside the event window that is not kernel execution totals 127.65 ms, 2.7% of wall time, plus 3.46 ms of D2H; even charging that entire residual to transfer, which overstates it, moves under 3% of the pipeline. The RTX 4090's residual is larger both absolutely and as a share of its window (152.14 ms, 36.5% against 22.1%), the direction PCIe versus NVLink-C2C predicts, though small enough that we do not lean on it. The one clean and clearly asymmetric transfer measurement is D2H.

What Table III shows instead is an Amdahl's Law [10] ceiling. Layer 8 is now 11.2% of GB10 wall time and 6.6% on the RTX 4090, while CPU Layers 1–7, still on 20 and 4 OpenMP threads rather than serial, account for 71.4% and 84.5%. Making Layer 8 free would still leave the RTX 4090 pipeline at 93% of current wall time, so offloading Layers 1–7 is the only remaining path to sub-second reconstruction; a Roofline analysis [11] would establish whether that offload is compute- or memory-bound first. The unaccounted remainder is process launch, file I/O and the per-frame `uint8`↔`double` conversions, all single-threaded and outside the instrumented region.

VI. Cross-Platform Validation on Discrete-GPU Hardware

To test whether the approach generalizes beyond unified memory, we replicated the experiment on a discrete-GPU desktop (Table I). Two differences matter: CPU and GPU memory are separate over PCIe rather than sharing one pool, and the CPU is heterogeneous (24 physical / 32 logical threads across P- and E-cores) against the GB10's 20 homogeneous-per-tier cores with no SMT.

A. Methodology Adjustment and GPU Speedup

Reusing the GB10's ladder and 20-thread backdrop misleads here: wall time collapses past 8 threads to ∼3.5× slower than the 4-thread optimum, reproducibly across three full runs. We therefore swept a topology-correct ladder (1–32) and used this platform's own best CPU configuration (4 threads) as backdrop. The collapse is consistent with the P/E-core split relying on Intel Thread Director while the generic OpenMP pool carries no affinity control [5], though the 4-thread optimum does not cleanly match the 8 P-cores, so we claim no confirmed explanation (Section VII). It is incidental to our contribution, reported because it changes what a fair CPU baseline is here. With the backdrop corrected, GPU V2 reaches 5,435–5,579 ms (26.9–27.6 FPS), 1.59×–1.63× over CPU V1's best (4 threads, 8,844.17 ms). That is not the inflated 4×+ obtained by comparing against serial time, or against a collapsed high-thread-count run such as CPU V0 at 32 threads (Table II).

B. Cross-Architecture Bit-Exactness

Section IV-A establishes bit-exactness *within* each platform. Whether GPU V2's output is identical *across* them is a stronger question. The GB10 (ARM host, `sm_121`) and the RTX 4090 (x86 host, `sm_89`) differ in host ISA and GPU microarchitecture, so bit-identical output is not guaranteed by IEEE-754 `double` compliance alone: code generation for two SM targets, FMA contraction, and math libraries can each diverge in the last bit. The toolchains are otherwise closely matched (GCC 13.3.0 and CUDA 13.0 on both), so this isolates ISA and microarchitecture rather than compounding them with a toolkit difference; a cross-*toolkit* claim would need its own experiment. Running the winning 32×8 configuration on both, `cmp -l` reports 0 differing bytes and the SHA-256 digests match (`8db2c07a...ca066b`). That is the strongest form of the determinism claim this paper makes.

C. Warp Alignment Generalizes Across GPU Architectures

At the kernel level 32×8 remains fastest on this Ada Lovelace GPU (360.92 ± 7.22 ms), 17.4% faster than 16×16 (437.06 ± 3.86 ms), confirming that the coalescing argument of Section III-B, a consequence of CUDA's 32-thread warp granularity [6], is a property of CUDA rather than of the GB10. The localization holds too: `deconv_kernel` falls 15.6% (310.32 to 261.90 ms) while the reduction is unchanged at 2.62 ms. Fig. 2 compares all five grid configurations on both GPUs, with error bars over the six runs. At the *wall-clock* level the five are statistically hard to distinguish (5,435–5,579 ms, overlapping error bars) even though 32×8 nominally wins both metrics; the kernel-level advantage is visible only through CUDA-event profiling, which is why that instrumentation matters.

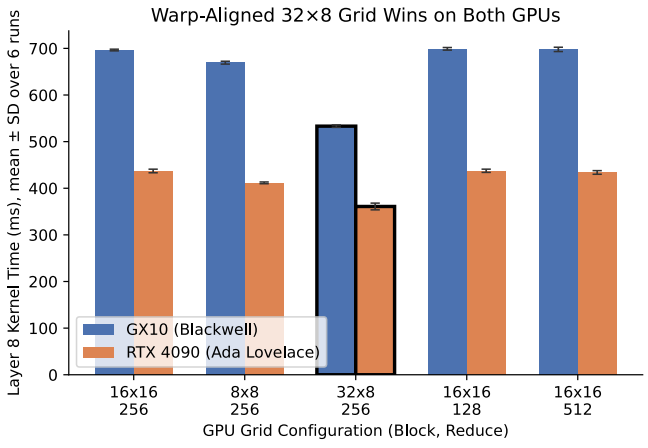

Fig. 2. GPU Layer 8 kernel time by grid configuration, mean ± SD over 6 runs. The warp-aligned 32×8 grid (outlined) is fastest on both architectures.

D. Non-Unified Memory and FP64 Precision

CPU and GPU memory here are physically separate over PCIe, unlike the GB10's unified pool [20], [21]. Even so,

this platform's Layer 8 kernel is the faster of the two. Table III attributes that to the compute and cache advantages of Section IV-E rather than to any absence of transfer cost: PCIe/DMA overhead [7] is plainly visible in the residual and the D2H figure, but it is outweighed. Double precision also costs here. Its FP64 throughput is architecturally $1/64$ of its FP32, so the kernel time above is achieved with the dominant compute resource idle, and an FP32 port would plausibly be much faster at the price of weakening bit-exactness to a numerical tolerance.

VII. Limitations and Future Work

The largest gap is the absence of a competitive GPU baseline: V2 is compared only against CPU implementations, not `atomicAdd`, an output-stationary gather kernel, or cuDNN. Two of our explanations are inferences from timing rather than direct measurements. The cache account of the $10\times$ reduction gap would be settled by L2 hit-rate counters and a working-set sweep across the L2 boundary; the coalescing account, though localized to `deconv_kernel` as predicted, would be settled by load-efficiency counters and non-aligned block widths. Our host-to-device figure also remains an upper bound containing launch overhead and host padding, separable with pinned memory or multiple streams, though D2H is unaffected. No FP32 ablation quantifies the speedup-versus-bit-exactness trade-off, and no `OMP_PROC_BIND` control isolates the RTX 4090 collapse from bandwidth saturation or thermal throttling. Finally, all results use one 150-frame QCIF-to-CIF sequence, with reconstruction quality (Section IV-C) measured at a single scale factor against a bicubic-downsampled stand-in and chroma replicated rather than reconstructed. Higher resolutions would shift the compute-to-transfer ratio and push the private buffer past the RTX 4090's L2, the regime in which our design rule is most testable.

VIII. Conclusion

We introduced a GPU spatial-reduction framework for FSRCNN's Layer 8 deconvolution and evaluated it on two architecturally distinct platforms. On both, the CPU and GPU variants are bit-exact against a verified deterministic reference at every configuration tested, while naive accumulation corrupts output non-monotonically, falling as low as 13.8 dB and reaching a near-undetectable 68.6 dB, the more instructive failure. GPU output is further bit-identical *across* the two platforms, spanning two host ISAs and two GPU generations. The warp-aligned 32×8 grid is fastest on both, with per-kernel profiling localizing the entire gain to the deconvolution as the coalescing explanation requires, and speedups over each platform's best correctness-verified CPU configuration reach $1.46\times$ (31.5 FPS) and $1.63\times$. The two platform asymmetries run in opposite directions. Unified memory returns results $4.74\times$ faster, while the discrete GPU runs the bandwidth-bound reduction $10.0\times$ faster because its L2 holds the entire private buffer that the GB10 must stream at 95% of memory peak. The cost of determinism therefore depends less on the algorithm than on whether the privatization working set fits last-level cache, a boundary that should transfer to any network to which this pattern is applied.

References

[1] Annisa, Adnan, and Z. Zainuddin, "Enhancing computational efficiency: transitioning from serial to parallel programming for low-to-high resolution video reconstruction using FSRCNN on AMP architecture," in *Proc. IEEE Int. Conf. Artif. Intell. Mechatron. Syst. (AIMS)*, 2025, pp. 1–8.

[2] C. Dong, C. C. Loy, and X. Tang, "Accelerating the super-resolution convolutional neural network," in *Proc. ECCV*, 2016, pp. 391–407.

[3] S. Wang *et al.*, "High-throughput CNN inference on embedded ARM big.LITTLE multicore processors," *IEEE TCAD*, vol. 39, no. 10, pp. 2254–2267, 2020.

[4] C. Bilbao, J. C. Saez, and M. Prieto-Matías, "Flexible system software scheduling for asymmetric multicore systems with PMCSched: a case for Intel Alder Lake," *Concurr. Comput. Pract. Exper.*, vol. 35, no. 25, e7814, 2023.

[5] A. E. Eichenberger *et al.*, "The design of OpenMP thread affinity," in *IWOMP*, LNCS 7312, 2012, pp. 15–28.

[6] NVIDIA Corp., *CUDA C++ Programming Guide*. [Online]. Available: docs.nvidia.com/cuda/cuda-c-programming-guide/

[7] Y. Go *et al.*, "APUNet: revitalizing GPU as packet processing accelerator," in *Proc. USENIX NSDI*, 2017, pp. 83–96.

[8] Z. Wang *et al.*, "Image quality assessment: from error visibility to structural similarity," *IEEE Trans. Image Process.*, vol. 13, no. 4, pp. 600–612, 2004.

[9] C. Dong *et al.*, "Learning a deep convolutional network for image super-resolution," in *Proc. ECCV*, 2014, pp. 184–199.

[10] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," in *Proc. AFIPS*, 1967, pp. 483–485.

[11] S. Williams, A. Waterman, and D. Patterson, "Roofline: an insightful visual performance model for multicore architectures," *Commun. ACM*, vol. 52, no. 4, pp. 65–76, 2009.

[12] S. Chetlur *et al.*, "cuDNN: efficient primitives for deep learning," arXiv:1410.0759, 2014.

[13] K. Serebryany and T. Iskhodzhanov, "ThreadSanitizer: data race detection in practice," in *Proc. WBIA*, 2009, pp. 62–71.

[14] A. Lashgar, A. Baniasadi, and A. Khonsari, "Warp size impact in GPUs: large or small?," in *Proc. GPGPU-5*, 2012, pp. 27–32.

[15] V. Dumoulin and F. Visin, "A guide to convolution arithmetic for deep learning," arXiv:1603.07285, 2016.

[16] NVIDIA Corp., "NVIDIA puts Grace Blackwell on every desk and at every AI developer's fingertips," NVIDIA Newsroom, Jan. 2025. [Online]. Available: nvidianews.nvidia.com/news/nvidia-puts-grace-blackwell-on-every-desk-and-at-every-ai-developers-fingertips

[17] ASUSTeK Computer Inc., *ASUS Ascent GX10 Datasheet*, v8, 2025. [Online]. Available: dlcdnwebimgs.asus.com/files/media/202506/5c0fb57c-4e48-4e96-8c97-04bf8df2677c/asus-ascent-gx10-datasheet.pdf

[18] Arm Ltd., "DynamIQ technology." [Online]. Available: arm.com/technologies/dynamiq

[19] NVIDIA Corp., *NVIDIA Ada GPU Architecture*, Whitepaper V2.02, 2022.

[20] NVIDIA Corp., "Grace Hopper Superchip architecture in-depth," NVIDIA Developer Blog, 2022.

[21] J. Wahlgren *et al.*, "Dissecting CPU-GPU unified physical memory on AMD MI300A APUs," in *Proc. IEEE IISWC*, 2025.